Stage 1: Project Planning & Management

ML Diploma Graduation Project | Deliverable Template v1

Team name	Team 171
Project title	AI-Powered Predictive Maintenance for Industrial Equipment
Submission date	16th October, 2026
Version	v1

1. Project Summary

Industrial equipment that fails unexpectedly causes costly downtime, and maintenance teams need early warning to schedule repairs before a breakdown. We will build a machine-learning system on the NASA C-MAPSS FD001 engine sensor dataset that classifies whether an engine is likely to fail within the next 30 operational cycles, with a secondary regression model for Remaining Useful Life (RUL) if it can be completed without delaying the core system. We will compare Random Forest, XGBoost, and LSTM models on an engine-based split, then connect the selected model to a FastAPI service and a Streamlit dashboard, with MLflow tracking experiments and model versions and Docker packaging for a local deployment. A live-style demo will replay a held-out engine's sensor readings cycle by cycle. The system is intended for maintenance engineers and planners who need to monitor engine health and prioritize inspections.

2. Team and Roles

Member name	Email	Primary role	Main responsibilities
Malak Mohamed Elsaid	malakelfarash@gmail.com	Team Lead; Project Coordination & Data Engineering	Planning, GitHub workflow, dataset setup, architecture, and integration. Single point of contact with the instructor.
Sandra Sameh Saad	sandrasameh248@gmail.com	Data Analysis & Feature Engineering	EDA, data cleaning, sensor visualizations, and time-series features.
Razan Ahmed Ramadan	razanahmedramadan@gmail.com	Machine Learning	Target preparation, Random Forest and XGBoost, tuning, and model comparison.
Dareen Mohamed Elsayed	es-dareen.mohamed2027@alexu.edu.eg	Time-Series & Evaluation	LSTM, sequence preparation, validation, metrics, and error analysis.
Jana Ahmed Farouk	es-jana.ahmed2027@alexu.edu.eg	Deployment & MLOps	FastAPI, Streamlit, MLflow, Docker, logging, and basic monitoring.

3. Objectives and Success Criteria

ID	Objective	How it is measured	Target
O-01	Failure classifier: predict whether an engine will fail within the next 30 cycles.	Precision, Recall, F1-score, and ROC-AUC on held-out test engines	Recall ≥ 0.85, F1 ≥ 0.80, ROC-AUC ≥ 0.90
O-02	Model comparison: Random Forest, XGBoost, and LSTM, with the best model selected.	All three evaluated on the same engine-based train/validation/test split	All 3 models on all 4 metrics; selected model beats the simplest baseline on validation F1
O-03	RUL estimation (secondary), started only if it does not delay O-01, O-02, and O-04.	MAE and RMSE on held-out test engines	RMSE ≤ 30 cycles (stretch: MAE ≤ 25 cycles)
O-04	Working application: FastAPI service and Streamlit dashboard in Docker, with a simulated live replay of a held-out engine.	Run from the README with Docker; replay demo run end to end	Starts from the README; replay runs with no manual fixes
O-05	Reproducibility: experiments, parameters, metrics, and model versions logged, plus prediction logging and basic drift checks.	Share of reported results traceable to an MLflow run	100% of reported results

4. Scope

In scope	Out of scope
NASA C-MAPSS FD001 data analysis, cleaning, and preprocessing	Other C-MAPSS subsets (FD002 to FD004)
30-cycle failure label and time-series feature engineering	Real sensor hardware or live data feeds (the demo is a simulated replay)
Random Forest, XGBoost, and LSTM comparison with an engine-based split	Cloud deployment (first target is a local/Docker application)
RUL regression (secondary, only if it does not delay the core system)	Production-grade monitoring or automated retraining (only basic drift checks and documented retraining steps)
FastAPI service, Streamlit dashboard, MLflow tracking, Docker packaging, tests, and documentation	Mobile app or user authentication

5. Work Breakdown Structure

ID	Task	Owner	Start	End	Depends on
T-01	Finalize scope, requirements, GitHub repository, task plan, and initial architecture	Malak	21st Sep	25th Sep	None

ID	Task	Owner	Start	End	Depends on
T-02	Load FD001; document engine IDs, cycles, operating settings, and sensor columns	Malak	21st Sep	25th Sep	T-01
T-03	EDA, data-quality checks, and sensor/degradation visualizations	Malak, Sandra	28th Sep	2nd Oct	T-02
T-04	Clean the data; create the 30-cycle failure label and RUL values	Sandra, Razan	5th Oct	9th Oct	T-03
T-05	Define the engine-based train/validation/test split and LSTM sequence format	Dareen	12th Oct	23rd Oct	T-02
T-06	Prepare the API/dashboard layout and basic MLflow setup	Jana	12th Oct	23rd Oct	T-01
T-07	Create rolling averages, sensor changes, trends, and other time-series features	Sandra	12th Oct	23rd Oct	T-04
T-08	Train and tune Random Forest and XGBoost for failure prediction	Razan	26th Oct	6th Nov	T-05, T-06, T-07
T-09	Prepare sequences and train the LSTM model	Dareen	26th Oct	6th Nov	T-04, T-05, T-06
T-10	Evaluate and compare models (Precision, Recall, F1, ROC-AUC, RUL metrics where used)	Razan, Dareen	9th Nov	13th Nov	T-08, T-09
T-11	Build the first FastAPI/Streamlit prototype and connect MLflow tracking	Jana	2nd Nov	13th Nov	T-06
T-12	Organize reusable code modules and integrate the data/model workflow	Malak	9th Nov	20th Nov	T-07, T-08
T-13	Tune and save the strongest model with its preprocessing pipeline	Razan	16th Nov	20th Nov	T-10
T-14	Finalize LSTM comparison, threshold selection, and error analysis	Dareen	16th Nov	20th Nov	T-10
T-15	Prepare final sensor/degradation visuals and feature importance	Sandra	16th Nov	20th Nov	T-10
T-16	Connect the final model to FastAPI and Streamlit; add prediction logging	Jana	16th Nov	27th Nov	T-11, T-13
T-17	Build the simulated live sensor replay for a held-out engine	Dareen, Jana	23rd Nov	27th Nov	T-16
T-18	Complete system diagrams, API documentation draft, test plan, and end-to-end integration	Malak	23rd Nov	27th Nov	T-12, T-16
T-19	Finalize MLflow tracking, logging, basic drift checks, and Docker packaging	Jana	23rd Nov	27th Nov	T-16

ID	Task	Owner	Start	End	Depends on
T-20	Document deployment and retraining steps	Malak, Jana	23rd Nov	4th Dec	T-19
T-21	Run model, API, integration, and UI tests; record fixes	Malak, Dareen	23rd Nov	4th Dec	T-17, T-18
T-22	Finalize EDA, feature-engineering, and model-results sections of the report	Sandra, Razan	23rd Nov	4th Dec	T-14, T-15
T-23	Assemble README, technical documentation, user guide, and final report	Malak	23rd Nov	4th Dec	T-20, T-22
T-24	Prepare slides, rehearse the demo, and review each member's contribution	All members	30th Nov	4th Dec	T-21, T-23

6. Milestones

Milestone	Date	Deliverable	Owner
Stage 1: Project Planning & Management	16th October, 2026	This document	Malak
Stage 2: Literature Review	16th October, 2026	Literature review document	Malak
Stage 3: Requirements Gathering	16th October, 2026	Requirements document	Malak
Stage 4: System Analysis & Design	6th November, 2026	Design document	Malak
Internal: data pipeline complete	23rd October, 2026	Cleaned FD001 data, preprocessing pipeline, targets, and data split	Sandra
Internal: baseline model trained	6th November, 2026	Trained Random Forest and XGBoost baselines with MLflow runs	Razan
Internal: feature freeze	27th November, 2026	Final model, integrated app, and replay demo. After this date the team only tests, fixes, and documents.	Jana & Dareen
Stage 5: Implementation	30th November, 2026	Repository + implementation document	Malak
Stage 6: Final presentation & Testing & Reports	4th December, 2026	Report, slides, demo	All members

7. Tools and Collaboration

Purpose	Tool	Link
Source code repository	GitHub	Repository
Task board	Airtable	Task Board
Shared documents	WhatsApp	Private Chat
Team communication	WhatsApp	Private Chat
Experiment tracking	MLflow (tracking and model registry, run locally or in Docker)	http://localhost:5000 when the project is running

8. Communication Plan

Meeting or report	Frequency	Participants and purpose
Team sync	Twice a week	All members: review progress against the WBS, unblock tasks, and plan the next days
Progress update to instructor	Every two weeks and at each stage deadline	Team Lead to instructor: status against the weekly deliverables, risks, and decisions needed
Code review	Every pull request	One other member reviews before merge to main
Pre-submission review	2 days before each stage deadline	All members. Check the document against the template checklist and assign final fixes.

9. Risk Register

Risk	Likelihood (L/M/H)	Impact (L/M/H)	Owner	Mitigation
Data access: the FD001 files are unavailable or in an unexpected format	L	H	Malak	Download and document the dataset in Week 1 and keep a shared copy with its source link and file checksums.
A member drops out or is unavailable	M	H	Malak	Code and documentation are shared through GitHub so responsibilities can be reassigned if a member becomes unavailable.
The model does not beat the baseline, or the LSTM underperforms	M	M	Razan	Use Random Forest/XGBoost as the baseline and fallback; select the final model on validation metrics; the LSTM is a comparison, not a dependency.

Risk	Likelihood (L/M/H)	Impact (L/M/H)	Owner	Mitigation
Compute limits slow LSTM training and tuning	M	M	Dareen	Use short sequence windows and early stopping, run heavy jobs on a shared GPU notebook environment, and keep CPU-friendly models as the fallback.
Integration delays between model, API, and dashboard	M	H	Jana	Build the API prototype in Week 8 with an early model, agree the model/preprocessing interface up front, and enforce the Week 11 feature freeze.
Data leakage inflates results	L	H	Dareen	Split by engine, fit scalers and features on training engines only, and review the split in code review.
RUL regression delays the core classification system	M	M	Malak	Treat RUL as secondary and start it only after O-01, O-02, and O-04 are on track.

10. Definition of Done

A task is done only when all of the following are true:

1. The code, notebook or document is committed to a feature branch and merged into main through a pull request that was reviewed and approved by one other member.

2. It runs from a clean clone by following the README (notebooks run top to bottom without errors, random seeds fixed, no hard-coded local paths).

3. Data rules are respected: splits are by engine, nothing is fitted on validation or test data, and the test set is used only for final evaluation.

4. Experiments are logged in MLflow with parameters, metrics and the data version.

5. It is documented (docstrings or comments, plus a short entry in the README or the relevant report section) and written documents follow the stage template and its checklist.

6. Deployment tasks run through the documented Docker command and pass the integration test.

7. The task is moved to Done on the task board with a link to the pull request or file.